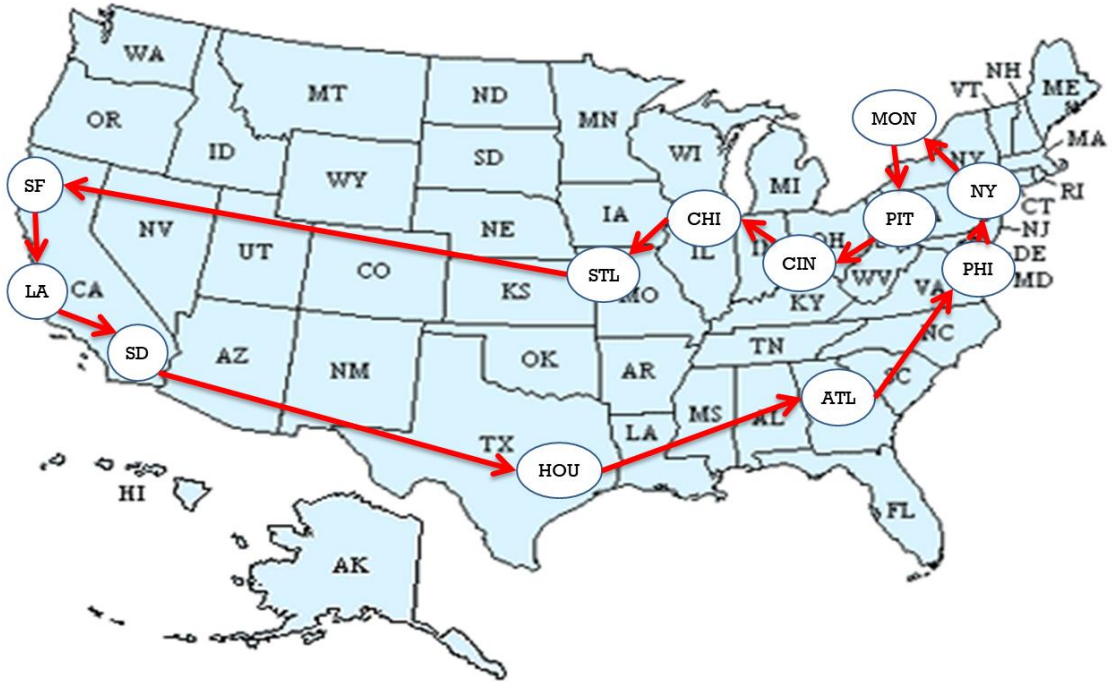
Data-Driven Optimization

7주차

Traveling Salesman Problem

Traveling Sales Man (TSP) problem

The travelling salesman problem (TSP) asks the following question: "**Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?**" It is an NP-hard problem in combinatorial optimization, important in operations research and theoretical computer science.



해집합의 크기 = (n-1)!

10! = 3,628,800
15! = 1,307,674,368,000
20! = 2,432,902,008,176,640,000

TSP in 데이터 기반 최적화

구분	주요 특징	적용 기술
파라미터 추정	정확한 데이터 확보 중심	회귀 분석, API 활용, 이력 데이터
확률적 대응	불확실성을 분포로 이해	마르코프 결정 과정 (MDP), 시뮬레이션
강건한 대응	최악의 상황 방어	불확실성 집합, 민감도 분석
동적 대응	실시간 변화 적응	실시간 재최적화, 메타휴리스틱

Test data for TSP

http://www.math.uwaterloo.ca/tsp/world/index.html

클래스넷에서 다운로드
tsp_data.zip

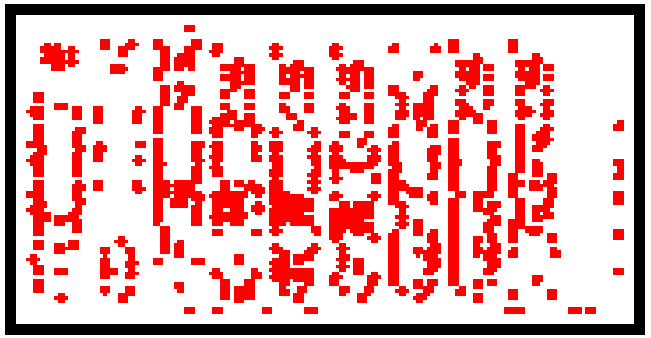
Milestones in the Solution of TSP Instances

Computer codes for the TSP have become increasingly more sophisticated over the years. A conspicuous sign of these improvements is the increasing size of nontrivial instances that have been solved, moving from Dantzig, Fulkerson, and Johnson's solution of a 49-city problem in 1954 up through the solution of a 24,978-city problem 50 years later.

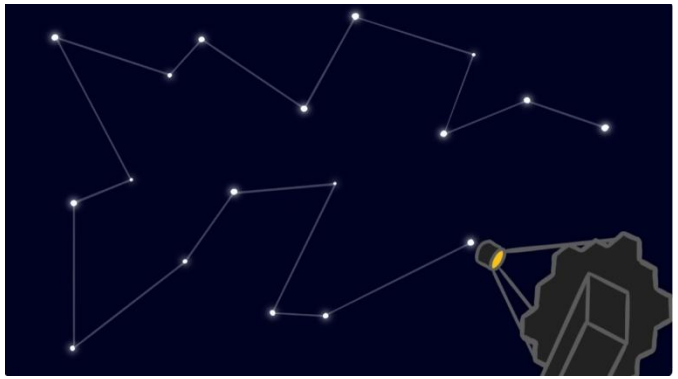
Year	Research Team	Size of Instance	Name
1954	G. Dantzig, R. Fulkerson, and S. Johnson	49 cities	dantzig42
1971	M. Held and R.M. Karp	64 cities	64 random points
1975	P.M. Camerini, L. Fratta, and F. Maffioli	67 cities	67 random points
1977	M. Grötschel	120 cities	gr120
1980	H. Crowder and M.W. Padberg	318 cities	lin318
1987	M. Padberg and G. Rinaldi	532 cities	att532
1987	M. Grötschel and O. Holland	666 cities	gr666
1987	M. Padberg and G. Rinaldi	2,392 cities	pr2392
1994	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	7,397 cities	pla7397
1998	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	13,509 cities	usa13509
2001	D. Applegate, R. Bixby, V. Chvátal, and W. Cook	15,112 cities	d15112
2004	D. Applegate, R. Bixby, V. Chvátal, W. Cook, and K. Helsgaun	24,978 cities	sw24798

Interesting problems

662 poins in VSLI (초고밀도 집적회로)

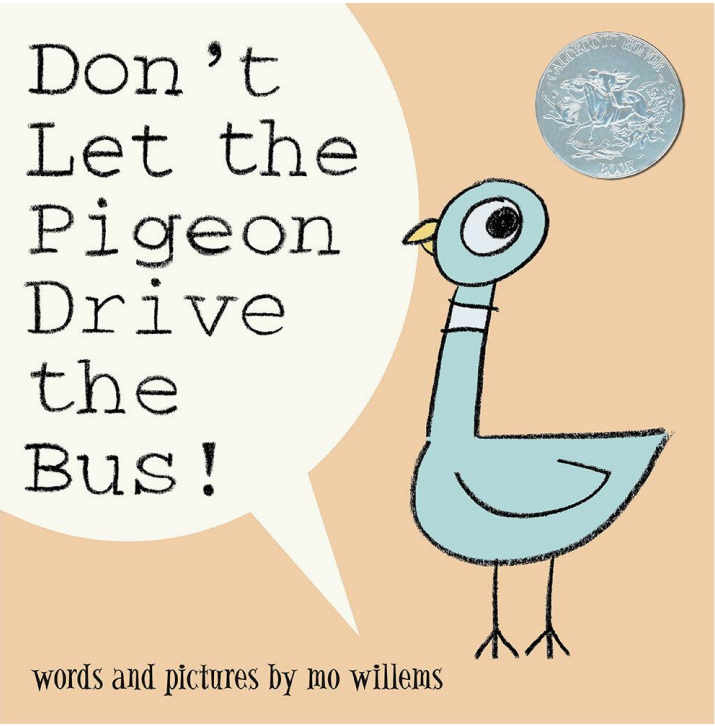


James Web 촬영 스케쥴



1,904,711-cities in the World

Interesting problems



Anim Cogn (2012) 15:379–391
DOI 10.1007/s10071-011-0463-9

ORIGINAL PAPER

Let the pigeon drive the bus: pigeons can plan future routes in a room

Brett Gibson · Matthew Wilkinson · Debbie Kelly

Received: 17 May 2011 / Revised: 5 September 2011 / Accepted: 13 September 2011 / Published online: 1 October 2011
© Springer-Verlag 2011

Abstract The task of determining an optimal route to several locations is called the *traveling salesperson problem* (TSP). The TSP has been used recently to examine spatial cognition in humans and non-human animals. It remains unclear whether or not the decision process of animals other than non-human primates utilizes rigid rule-based heuristics, or whether non-human animals are able to flexibly 'plan' future routes/behavior based on their knowledge of multiple locations. We presented pigeons in a One-way and Round-Trip group with TSPs that included two or three destinations (feeders) in a laboratory environment. The pigeons departed a start location, traveled to each feeder once before returning to a final destination. Pigeons weighed the proximity of the next location heavily, but appeared to plan ahead multiple steps when the travel costs for inefficient behavior appeared to increase. The results provide clear and strong evidence that animals other than primates are capable of planning sophisticated travel routes.

Keywords Spatial cognition · Navigation · Planning · Traveling salesperson · Pigeon

Introduction

In the beloved children's book—*Don't let the pigeon drive the bus!*—a human bus driver pleads with the reader not to 2003). On the one hand, the bus driver might have been concerned that the pigeon would not be able to safely drive the vehicle. On the other hand, maybe the driver was more concerned that the pigeon would not be able to take a route that would efficiently pick up all the passengers at the various stops throughout the city. For example, if the bus had to travel to three sequential stops that were on the same street, it would be more efficient to travel to the stops sequentially rather than to the first stop, followed by the third and then backtrack to return to the second stop before revisiting the third stop as the bus continued down the street.

The task of determining an optimal route to several locations or nodes is called the *traveling salesperson problem* (TSP). The number of possible solutions to a TSP that includes a round trip is defined as $(N - 1)!$, where N is the number of places to be visited. The ability to determine all possible solutions to find the most efficient route becomes more demanding as the number of locations (nodes) in the problem increases. With 5 locations, there are 12 possible routes, but with 10 locations, there are 181,440 possible travel routes! Thus, the task of finding an optimal route can have quite a cognitive demand and many "travelers" may resort to using less cognitively demanding strategies to reduce the task difficulty. The TSP has been used to examine spatial cognition in humans (MacGregor and Ormerod 1996; MacGregor et al. 1999, 2000; Vickers et al. 2006) and non-human animals (Gallistel and Cramer

Let the pigeon drive the bus:

여러 위치로 이동하는 최적의 경로를 결정하는 작업을 이동 영업사원 문제(TSP)라고 합니다. TSP는 최근 인간과 비인간 동물의 공간 인지를 조사하는 데 사용되었습니다. 인간 이외의 영장류가 아닌 동물의 의사 결정 과정이 엄격한 규칙 기반 휴리스틱을 사용하는지, 인간 이외의 동물이 여러 위치에 대한 지식을 바탕으로 미래의 경로/행동을 유연하게 '계획'할 수 있는지 여부는 아직 불분명합니다. 비둘기를 편도 및 왕복 그룹으로 나누어 실험실 환경에서 두세 곳의 목적지(먹이통)가 포함된 TSP를 제시했습니다. 비둘기들은 시작 지점을 출발하여 각 모이통으로 한 번씩 이동한 후 최종 목적지로 돌아오는 방식으로 진행되었습니다. 비둘기는 다음 장소의 근접성을 중요하게 고려했지만, 비효율적인 행동으로 인한 이동 비용이 증가하면 여러 단계를 미리 계획하는 것으로 나타났습니다. 이 결과는 영장류 이외의 동물도 정교한 이동 경로를 계획할 수 있다는 명확하고 강력한 증거를 제공합니다.

외판원 문제(Traveling Salesman Problem) 활용예

- 수리 방문 순서 결정
- 택배 차량 방문 순서
- 생산 설비에서 페인트 제품의 생산순서결정
- 집적 회로기판
- 천제물체 영상화

Traveling Salesman Problem

(의사결정변수)

x_{ij} : 도시 i에서 j를 방문하면 1,아니면 0

(목적함수)

$$Minimize \sum_{i=1}^{n} \sum_{j=1, j \neq i}^{n} c_{ij} x_{ij}$$ (도시 i에서 j를 방문하면 c_{ij} 비용이 발생)

(제약식)

$$\sum_{i=1, i \neq j}^{n} x_{ij} = 1$$ (각 도시 j는 한 번만 방문할 수 있다.) $j = 1, \ldots, n$

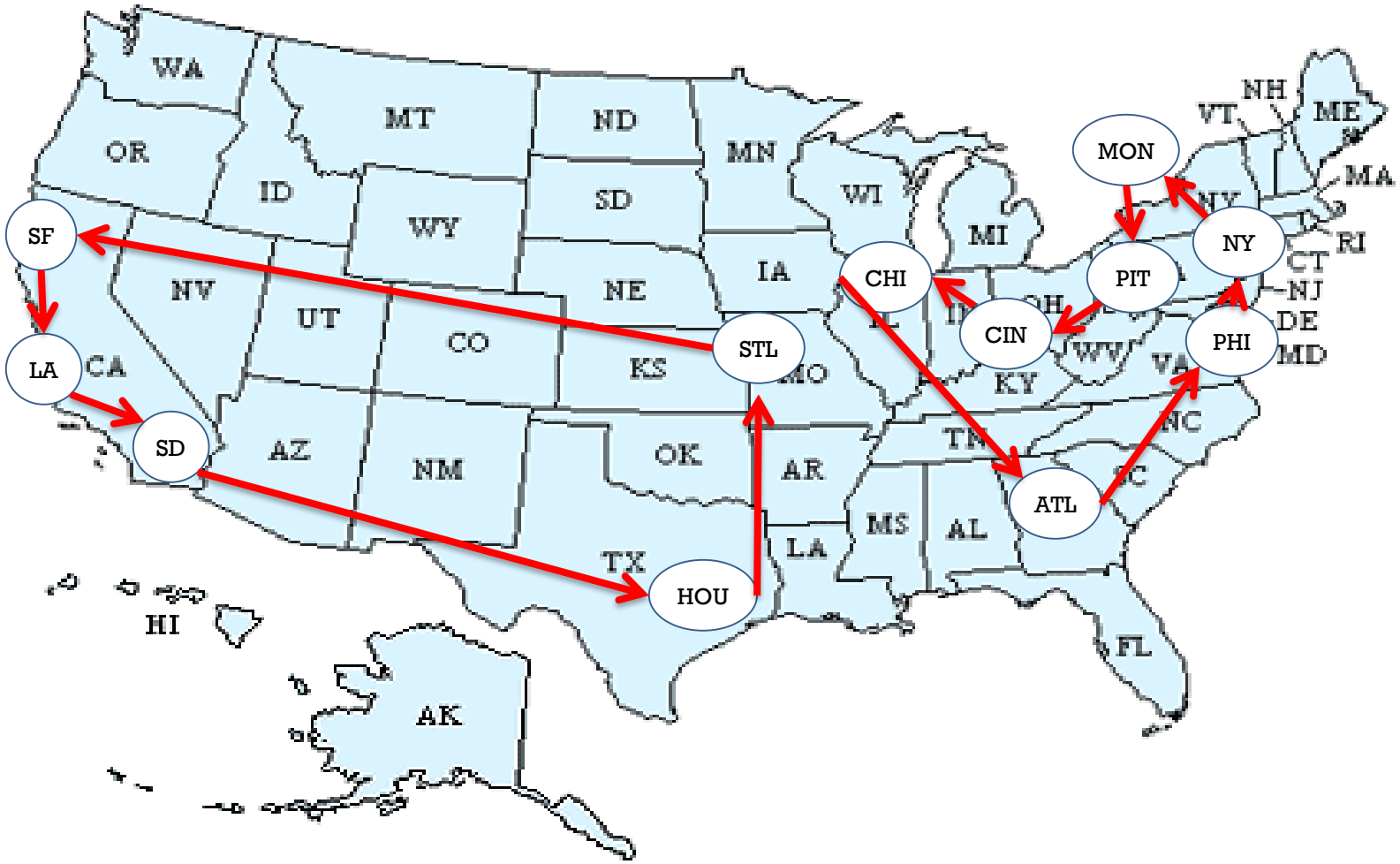
$$\sum_{j=1, j \neq i}^{n} x_{ij} = 1$$ (도시 j를 들어가면, 반드시 나와야 한다.) $i = 1, \ldots, n$

$$\sum_{i,j \in S}^{n} x_{ij} < |S| - 1 \quad for\ every\ subset\ S$$

Dantzig–Fulkerson–Johnson 공식

(출발 도시를 제외한 모든 부분집합 S에 대해 순환을 허용하지 않는다.)

TSP- 마지막 제약식이 없다면?



세번째 제약식

- **Dantzig–Fulkerson–Johnson 공식 : 모든 서브 셋을 찾고 그에 대해 다음 공식을 제약식으로 포함.**

$$\sum_{i,j \in S}^{n} x_{ij} < |S| - 1 \quad for\ every\ subset\ S$$

- **Miller–Tucker–Zemlin 공식 (구현이 더 쉬움) → 이 제약식을 사용함.**
  - 새로운 의사결정 변수와 제약 조건 도입

    (의사결정 변수) u_i, i도시의 방문순서, 실수 → 이해를 돕기 위해 일단 정수

    (제약조건)

$$u_i - u_j + nx_{ij} \leq (n-1) \quad 2 \leq i \neq j \leq n$$

$$2 \leq u_i \leq n, \qquad 2 \leq i \leq n$$

세번째 제약식 - 2

$$u_i - u_j + nx_{ij} \le (n-1) \quad 2 \le i \ne j \le n$$

$$2 \le u_i \le n, \qquad 2 \le i \le n$$

1) $x_{ij} = 1$ 이면 (도시 j 방문 순서가 도시 i 다음이면)

$\boldsymbol{u_i \le u_j - 1}$

$2 \le u_i \le n$ 에 의해 $u_i = u_j - 1$이 됨. (∵ n-1개 변수가 $2 \sim n$ 사이 값을 가져야 하므로)

2) $x_{ij} = 0$ 이면 (도시 j 방문 순서가 도시 i 다음이 아니면)

$u_i - u_j \le n - 1 \rightarrow \boldsymbol{u_i \le u_j + n - 1}$

u_i, u_j가 가질 수 있는 값은 $2 \sim n$ 이므로 두 변수의 최대차는 n-2이므로 n-1 보다 작다는 제약은 두 변수 값을 제약지 않음.

TSP 문제 – 제약식 설명 (예)

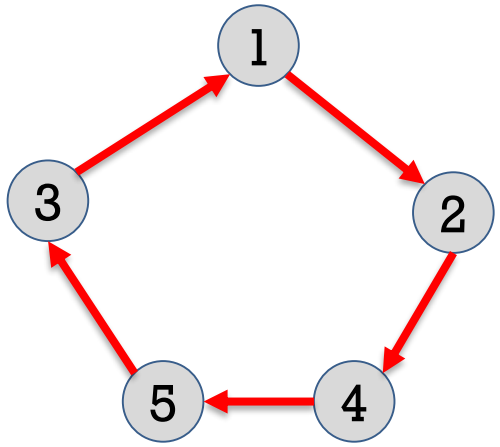
$$u_i - u_j + nx_{ij} \le (n-1) \quad 2 \le i \ne j \le n$$

$$2 \le u_i \le n, \qquad 2 \le i \le n$$

n = 5 일 때

i) $x_{ij} = 1$ ➔ $u_i \le u_j - 1$

ii) $x_{ij} = 0$ ➔ $u_i \le u_j + 4$



$x_{24} = 1 \Rightarrow u_2 \le u_4 - 1$
$x_{45} = 1 \Rightarrow u_4 \le u_5 - 1$
$x_{53} = 1 \Rightarrow u_5 \le u_3 - 1$
+
$2 \le u_2, u_3, u_4, u_5 \le 5$

➔ $u_2 = 2, u_3 = 5, u_4 = 3, u_5 = 4$

(그 외 제약은...)
$x_{42} = 0 \Rightarrow u_4 \le u_2 + 4$
$x_{54} = 0 \Rightarrow u_5 \le u_4 + 4$
$x_{35} = 0 \Rightarrow u_3 \le u_5 + 4$

예제 1

- Atlanta, Chicago, Cincinnati, Houston, LA, Montreal 6개 도시간 거리 정보가 다음과 같을 때 TSP 문제를 푸시오.

```
nCity = 6

DIST = [    [0, 702, 454, 842, 2396, 1196],
            [702, 0, 324, 1093, 2136, 764],
            [454, 324, 0, 1137, 2180, 798],
            [842, 1093, 1137, 0, 1616, 1857],
            [2396, 2136, 2180, 1616, 0, 2900],
            [1196, 764, 798, 1857, 2900, 0]
        ]
```

예제 1 - coding

```
from ortools.linear_solver import pywraplp

nCity = 6

DIST = [    [0, 702, 454, 842, 2396, 1196],
            [702, 0, 324, 1093, 2136, 764],
            [454, 324, 0, 1137, 2180, 798],
            [842, 1093, 1137, 0, 1616, 1857],
            [2396, 2136, 2180, 1616, 0, 2900],
            [1196, 764, 798, 1857, 2900, 0]
        ]

solver = pywraplp.Solver.CreateSolver("SAT")

X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))

U = {}
for i in range(1, nCity):
    U[i] = solver.IntVar(1, nCity-1, "U[%i]" %i)
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(i))

# 도시 j로 부터 한 번은 나와야 한다.
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))

# 방문 순서 제약
for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + 1 - (nCity-1)*(1-X[i,j]) <= 0, 'U_'+str(i)+str(j))

# Objective
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])
solver.Minimize(solver.Sum(objective_terms))
```

예제 1 - coding

```
if 1:
    with open(‘or7-1.lp', "w") as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)

# Solve
status = solver.Solve()

# Print solution.
if status == pywraplp.Solver.OPTIMAL or status == pywraplp.Solver.FEASIBLE:
    print(f"Total cost = {solver.Objective().Value():.1f}\n", )
    for i in range(nCity):
        for j in range(nCity):
            if i != j:
                if X[i, j].solution_value() > 0.5:
                    print(f"X{i} --> X{j}")
    for i in range(1, nCity):
        print(f"{i} 도시 방문순서: ", U[i].solution_value())
else:
    print("No solution found.")
```

답: 6610

실습 1

| 예제: 외판원 문제 |

현재 실업자인 로즈씨는 돈을 벌기 위해 다음과 같은 계획을 만들었다. 그는 열여덟 사람을 안내해서 내셔널 리그 모든 구장을 방문하는 것이다. 그는 이 안내일을 통해 돈을 벌고자 하기 때문에 되도록 순회 비용을 최소화하려고 한다. 신시내티에서 출발해서 신시내티로 돌아온다. 다음과 같은 거리 매트릭스가 만들어졌다.

	Atl	Chi	Cin	Hou	LA	Mon	NY	Phi	Pit	StL	SD	SF
아틀랜타(Atl)	0	702	454	842	2396	1196	864	772	714	554	2363	2679
시카고(Chi)	702	0	324	1093	2136	764	845	764	459	294	2184	2187
신시내티(Cin)	454	324	0	1137	2180	798	664	572	284	338	2228	2463
휴스턴(Hou)	842	1093	1137	0	1616	1857	1706	1614	1421	799	1521	2021
L.A.	2396	2136	2180	1616	0	2900	2844	2752	2464	1842	95	405
몬트리올(Mon)	1196	764	798	1857	2900	0	396	424	514	1058	2948	2951
뉴욕(NY)	864	845	664	1706	2844	396	0	92	386	1002	2892	3032
필라델피아(Phi)	772	764	572	1614	2752	424	92	0	305	910	2800	2951
피츠버그(Pit)	714	459	284	1421	2464	514	386	305	0	622	2512	2646
세인트루이스(StL)	554	294	338	799	1842	1058	1002	910	622	0	1890	2125
샌디에고(SD)	2363	2184	2228	1521	95	2948	2892	2800	2512	1890	0	500
샌프란시스코(SF)	2679	2187	2463	2021	405	2951	3032	2951	2646	2125	500	0

실습 1

tsp_ex_data1.xlsx

	A	B	C	D	E	F	G	H	I	J	K	L	M
1		ATL	CHI	CIN	HOU	LA	MON	NY	PHI	PIT	STL	SD	SF
2	ATL	0	702	454	842	2396	1196	864	772	714	554	2363	2679
3	CHI	702	0	324	1093	2136	764	845	764	459	294	2184	2187
4	CIN	454	324	0	1137	2180	798	664	572	284	338	2228	2463
5	HOU	842	1093	1137	0	1616	1857	1706	1614	1421	779	1521	2021
6	LA	2396	2136	2180	1616	0	2900	2844	2752	2464	1842	95	405
7	MON	1196	764	798	1857	2900	0	396	424	514	1058	2948	2951
8	NY	864	845	664	1706	2844	396	0	92	386	1002	2892	3032
9	PHI	772	764	572	1614	2752	424	92	0	305	910	2800	2951
10	PIT	714	459	284	1421	2464	514	386	305	0	322	2512	2646
11	STL	554	294	338	799	1842	1058	1002	910	622	0	1890	2125
12	SD	2363	2184	2228	1521	95	2948	2892	2800	2512	1890	0	500
13	SF	2679	2187	2463	2021	405	2951	3032	2951	2646	2125	500	0
14													

```
df = pd.read_excel('./tsp_ex_data1.xlsx', index_col = 'Unnamed: 0')
DIST = df.values.tolist()
```

실습 답: 7577

실습 2 - TSP_LIB 파일 읽어 TSP 문제 만들기

```
from ortools.linear_solver import pywraplp
import pandas as pd
import math
import matplotlib.pyplot as plt

def calDist(x1, y1, x2, y2):

    dist = math.sqrt((x1-x2)**2+(y1-y2)**2)

    return dist

def makeDIST(nP):

    DIST = list()
    nCity = len(nP)

    for i in range(nCity):
        DIST.append([])
        for j in range(nCity):
            if j != i:
                temp = calDist(nP['xc'][i], nP['yc'][i], nP['xc'][j], nP['yc'][j])
                DIST[i].append(temp)
            else:
                DIST[i].append(0)

    return DIST
```

```
#%% MAIN FLOW
nPos = pd.DataFrame(columns=[ 'xc', 'yc'])

f = open(".\\Tsp_data\\bays29.tsp", 'r')
#f = open(".\\Tsp_data\\berlin52.tsp", 'r')

flag = 0

while True:
    line = f.readline().strip()

if not line: break

    #if line == "DISPLAY_DATA_SECTION":
    if line == "NODE_COORD_SECTION":
        flag = 1
        continue

    if line == "EOF":
        continue

    if flag:
        ss = line.split()
        nPos.loc[len(nPos)] = [float(ss[1]), float((ss[2]))]

DIST = makeDIST(nPos)
f.close()

#%%
plt.scatter(nPos['xc'], nPos['yc'])
plt.show()
```

실습 2 - TSP_LIB 파일 읽어 TSP 문제 만들기

```python
#%% MAIN FLOW

nPos = pd.DataFrame(columns=[ 'xc', 'yc'])

f = open(".\\Tsp_data\\bays29.tsp", 'r')
#f = open(".\\Tsp_data\\berlin52.tsp", 'r')

flag = 0

while True:
    line = f.readline().strip()

    #print(line)

    if not line: break

    if line == "DISPLAY_DATA_SECTION":
    #if line == "NODE_COORD_SECTION":
        flag = 1
        continue

    if line == "EOF":
        continue

    if flag:
        ss = line.split()
        nPos.loc[len(nPos)] = [float(ss[1]), float((ss[2]))]

DIST = makeDIST(nPos)
final_sol = []

print(DIST)

f.close()
```

답: 9074.1

실습 3번 – 옵션 방문 TSP

(의사결정변수)

x_{ij} : 도시 i에서 j를 방문하면 1,아니면 0

$\boldsymbol{y_i}$ **: 도시 i를 방문하면 1,아니면 0**

u_i : 도시 i의 방문순서

(목적함수)

$$Minimize \sum_{i=1}^{n} \sum_{j=1, j \neq i}^{n} c_{ij} x_{ij} - \sum_{j=1, j \neq i}^{n} \boldsymbol{v_i y_i}$$ (도시 i에서 j를 방문하면 c_{ij} 비용이 발생하고 도시 i를 방문하면 vi의 수익이 발생)

(제약식)

$$\sum_{i=1, i \neq j}^{n} x_{ij} = y_i$$ (각 도시 j는 기껏해야 한 번만 방문할 수 있다.) $j = 1, \ldots, n$

$$\sum_{j=1, j \neq i}^{n} x_{ij} = y_i$$ (도시 j를 들어가면, 반드시 나와야 한다.) $i = 1, \ldots, n$

$$u_i - u_j + n x_{ij} \leq (n-1) \quad 2 \leq i \neq j \leq n, \qquad 2 \leq u_i \leq n, \qquad 2 \leq i \leq n$$

$$\sum_{j=1}^{n} \boldsymbol{y_i} = \boldsymbol{N}$$ **(N 개 도시만 방문)**

실습 3

| 예제: 외판원 문제 |

현재 실업자인 로즈씨는 돈을 벌기 위해 다음과 같은 계획을 만들었다. 그는 열여덟 사람을 안내해서 내셔널 리그 모든 구장을 방문하는 것이다. 그는 이 안내일을 통해 돈을 벌고자 하기 때문에 되도록 순회 비용을 최소화하려고 한다. 신시내티에서 출발해서 신시내티로 돌아온다. 다음과 같은 거리 매트릭스가 만들어졌다.

	Atl	Chi	Cin	Hou	LA	Mon	NY	Phi	Pit	StL	SD	SF
아틀랜타(Atl)	0	702	454	842	2396	1196	864	772	714	554	2363	2679
시카고(Chi)	702	0	324	1093	2136	764	845	764	459	294	2184	2187
신시내티(Cin)	454	324	0	1137	2180	798	664	572	284	338	2228	2463
휴스턴(Hou)	842	1093	1137	0	1616	1857	1706	1614	1421	799	1521	2021
L.A.	2396	2136	2180	1616	0	2900	2844	2752	2464	1842	95	405
몬트리올(Mon)	1196	764	798	1857	2900	0	396	424	514	1058	2948	2951
뉴욕(NY)	864	845	664	1706	2844	396	0	92	386	1002	2892	3032
필라델피아(Phi)	772	764	572	1614	2752	424	92	0	305	910	2800	2951
피츠버그(Pit)	714	459	284	1421	2464	514	386	305	0	622	2512	2646
세인트루이스(StL)	554	294	338	799	1842	1058	1002	910	622	0	1890	2125
샌디에고(SD)	2363	2184	2228	1521	95	2948	2892	2800	2512	1890	0	500
샌프란시스코(SF)	2679	2187	2463	2021	405	2951	3032	2951	2646	2125	500	0

추가로, 각 도시를 방문했을 때 얻는 기대 수익은 profits = [35000, 13000, 58000, 41000, 13000, 54000, 65000, 48000, 69000, 56000, 34000, 67000] 와 같고 이 도시들 중 6개만 방문할 수 있다고 하고 TSP 문제를 풀어라.

실습

- 모델링
- 예제 1
- 실습 1, 2, 3
- Tsp_data/berlin52.tsp 에 대한 TSP 최적화 python 프로그램을 작성

모델링

- 어떤 영업사원은 하루에 4개 지역을 순회하는데 전체 이동거리가 최소가 되는 경로를 알고 싶다. 4개 지역들 간의 거리는 다음과 같다.

도시	1	2	3	4
1	0	13	25	15
2	13	0	-	21
3	25	26	0	11
4	15	-	11	0

- 도시 i에서 도시 j로 이동하면 $x_{ij}=1$, 아니면 0이라고 하고 u_i를 도시 i의 방문순서라고 할 때 이 문제에 대한 정수모형을 작성하여라.